



OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG

ELEKTRO- UND
INFORMATIONSTECHNIK

PRAKTIKUM INFORMATIK 1

M. Niemetz

Algorithmen

Flächenberechnung mit Monte-Carlo-Integration

Mai 2025, Version: 3.1

Fakultät Elektro- und Informationstechnik

1 Einleitung

Allmählich haben Sie die meisten der elementaren Programmierwerkzeuge kennengelernt und die Frage, wie man nun damit konkrete Probleme löst, rückt in das Zentrum Ihrer Ausbildung.

Hierfür gibt es eine Vielzahl raffinierter Vorgehensweisen (Algorithmen), die zu kennen einem die Ideen an die Hand gibt, die helfen, das eigene Programmierproblem zu lösen.

In dieser Aufgabe lernen Sie als Beispiel für eine Anwendung des Zufalls in der Entwicklung von Algorithmen die Monte-Carlo-Integration kennen.

Monte-Carlo-Algorithmen haben ihren Namen erhalten, weil die Zielerreichung des Algorithmus auf statistische Methoden setzt, d.h. anstatt das Ergebnis wie üblich durch immer feinere Auflösung (also zum Beispiel immer kleinere Schritte bei einer Riemannschen Integration, wie Sie sie in der Mathematik kennengelernt haben) an den tatsächlichen Wert anzunähern, wird bei den Monte-Carlo-Algorithmen das Ergebnis gewissermaßen zufällig ausgewürfelt.

Diese Algorithmen werden häufig für komplexe Probleme eingesetzt, welche sich nicht geschlossen lösen lassen oder bei denen konventionelle Algorithmen zu ineffizient wären, um in endlicher bzw. akzeptabler Zeit berechnet zu werden (zum Beispiel bei Problemen in vielen Dimensionen).

Die Monte-Carlo-Algorithmen sind damit eine Klasse von zufallsbasierten Algorithmen, welche mit einer endlich (kleinen) Wahrscheinlichkeit auch ein falsches Ergebnis bzw. eine Näherungslösung liefern dürfen. Darin unterscheiden sie sich von den (weniger populären) Las-Vegas-Algorithmen, welche nach Ihrer vollständigen Abarbeitung stets ein korrektes Ergebnis liefern müssen. Interessierte finden weiterführende Literatur zum Beispiel in [1].

Die Monte-Carlo-Algorithmen werden in der Regel so eingesetzt, dass ein Problem näherungsweise gelöst wird und stehen damit in Konkurrenz zu mehreren anderen Algorithmenfamilien.

Je nach der Beschaffenheit der zu bewältigenden Problematik muss der Algorithmus passend gewählt werden, da sich die Algorithmen in ihren Eigenschaften unterscheiden, wie zum Beispiel dem Rechenaufwand, dem Speicherverbrauch der Konvergenzgeschwindigkeit pro Rechenaufwand oder der erreichbaren Präzision.

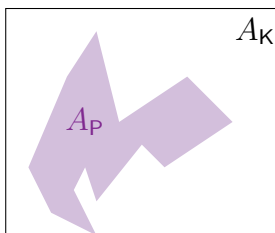
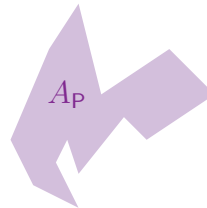
In der vorliegenden Aufgabe soll als relativ einfaches Beispiel ein Flächenintegral numerisch bestimmt werden, wobei die Umrandung der Fläche durch ein beliebiges überschneidungsfreies Polygon gegeben ist.

Im Folgenden wird nun der Algorithmus und dessen Umsetzung erarbeitet.

2 Berechnung einer Polygonfläche

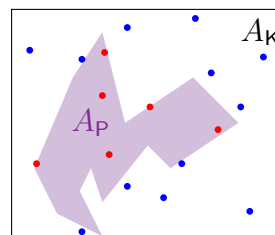
2.1 Die Idee

Nebstehend sehen Sie als Beispiel ein Polygon, dessen von ihm eingeschlossene Fläche ermittelt werden soll. Die Idee zur Berechnung einer Polygonfläche A_P mit einem Monte-Carlo-Algorithmus ist geradezu atemberaubend einfach.



Man wähle einen beliebigen Bereich A_K der Gesamtebene, welcher die zu berechnende Fläche komplett einschließt¹ und dessen Fläche leicht zu berechnen ist, zum Beispiel ein Rechteck.

Dann wähle man **zufällig**² eine endliche Menge M_P von Punkten P_i aus diesem Bereich A_K und ermittle dann für jeden dieser Punkte, ob er innerhalb der zu berechnenden Fläche liegt („Treffer“) oder außerhalb („kein Treffer“): $M_P = M_{\text{Treffer}} \cup M_{\text{keinTreffer}}$



Da man die Punkte zufällig gewählt hat, muss das Verhältnis der Anzahl der Treffer $\#M_{\text{Treffer}}$ zur Gesamtpunktzahl $\#M_P$ dem Flächenverhältnis der „getroffenen“ Polygonfläche A_P zur Gesamtfläche A_K entsprechen.

Aus dem Anteil der Treffer an der Gesamtmenge der Punkte M_P und der Fläche A_K kann damit die gesuchte Fläche leicht durch einen Dreisatz berechnet werden:

$$A_P = A_K \cdot \frac{\#M_{\text{Treffer}}}{\#M_{\text{Treffer}} + \#M_{\text{keinTreffer}}}$$

¹Für die Effizienz und Genauigkeit des Verfahrens ist es hilfreich, diesen Bereich nicht unnötig groß zu wählen.

²Die zufällige Wahl der Punkte ist entscheidend für den Erfolg, da damit vermieden wird, dass die Fläche durch systematische Fehler falsch eingeschätzt wird, wie dies bei einem regelmäßigen oder sonst irgendwie festgelegten Raster der Fall sein könnte.

2.2 Der Algorithmus

Für die Umsetzung dieser Idee sind jetzt noch einige Detailprobleme zu lösen.

2.2.1 Festlegen einer umschließenden Fläche

Damit der Flächeninhalt der umschließenden Fläche möglichst leicht zu berechnen ist, wählen wir einen rechteckigen Ausschnitt.

Dabei verwenden wir die maximalen und minimalen x- bzw. y-Koordinaten der Eckpunkte des Polygonzugs. Dadurch wird das Rechteck minimal groß und wir erreichen eine optimale Genauigkeit.

Frage: Überlegen Sie sich, wie Sie aus eine Liste von x- bzw. y-Koordinaten das jeweilige Maximum bzw. Minimum der x- bzw. y-Koordinaten herausfinden können.

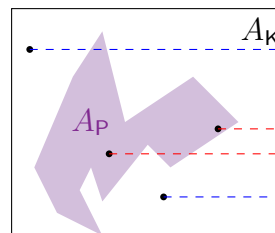
2.2.2 Drinnen oder draußen?

Die noch verbleibende und schwierigste Aufgabe ist es, für die Testpunkte $P_i \in M_P$ zu entscheiden, ob sie außerhalb oder innerhalb des Polygonzugs γ liegen der durch das Verbinden der Eckpunkte des Polygons entsteht.

Hierfür soll der sogenannte Test nach Jordan³ [2, 3] zum Einsatz kommen. Die grundsätzliche Vorgehensweise hierfür ist die, dass von einem Punkt, für den geprüft werden soll, ob er im Inneren des Polygonzugs liegt, ein „Suchstrahl“ in eine beliebige Richtung geschickt und dabei geprüft wird, wie oft dieser die Umrandung des Polygonzugs quert. Aus Gründen der Einfachheit und ohne Einschränkung der Allgemeinheit verwenden wir im folgenden einen Suchstrahl, der vom pzu prüfenden Punkt horizontal nach rechts verläuft.

Bei Punkten innerhalb des Polygons wird sich eine ungerade Anzahl von Überquerungsereignissen ergeben, bei Punkten außerhalb des Polygons eine gerade Anzahl oder Null.

Berücksichtigt man bei der Zählung der Schnittpunkte die Richtung des Überquerens (von links nach rechts oder umgekehrt – wobei man die Kanten des Polygons vom Startpunkt bis zum Endpunkt nacheinander in *einer* Richtung durchläuft) als Vorzeichen, so ergibt sich für Punkte außerhalb des Polygons immer Null als Summe, für solche innerhalb des Polygons ein Wert ungleich Null. Mit dieser Methode können auch Polygonzüge verarbeitet werden, die sich selbst überlappen.

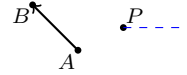


³Die Methode fußt auf dem Jordanschen Kurvensatz [4].

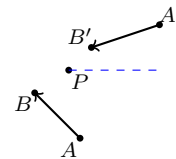
Bei der praktischen Umsetzung dieses Jordan-Tests ergeben sich verschiedene Fälle, die jeweils korrekt behandelt werden müssen.

Man findet durch Überlegen folgende Fälle für den Test eines Punktes P mit einem einzelnen, gerichteten Segment des Polygons $\overrightarrow{AB}$ begrenzt durch die Eckpunkte A und B :

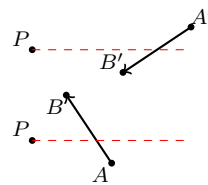
Beide Endpunkte A und B des Polygonsegments liegen links vom Testpunkt P : Es kann keinen Schnittpunkt geben, der Testwert für dieses Segment ist Null.



Beide Endpunkte A und B des Polygonsegments liegen oberhalb oder unterhalb des Testpunkts P : Es kann keinen Schnittpunkt geben, der Testwert für dieses Segment ist Null.

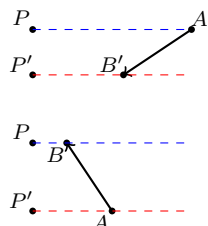


In den übrigen Fällen blickt man vom Punkt $A = (x_A, y_A)$ der gerichteten Strecke $\overrightarrow{AB}$ auf den Punkt $P = (x_P, y_P)$ und ermittelt, ob dieser „rechts“ oder „links“ vom Punkt $B = (x_B, y_B)$ zu sehen ist. Hierfür berechnet man das Kreuzprodukt (genauer: seine Z-Komponente, denn die anderen Komponenten sind Null weil sich alle Punkte in der $Z=0$ -Ebene befinden)



$$\begin{aligned} K &= (\vec{P} - \vec{A}) \times (\vec{B} - \vec{A}) \\ &= (x_P - x_A)(y_B - y_A) \\ &\quad - (x_B - x_A)(y_P - y_A). \end{aligned}$$

Dabei kommt es zu einer Überquerung des Suchstrahls für **abwärts** laufende Segmente ($y_A > y_B$), wenn $K > 0$, also der Punkt P von A aus gesehen rechts von der Strecke liegt: $\Rightarrow$ Testwert ist 1, bzw. für **aufwärts** laufende Segmente ($y_A < y_B$), wenn $K < 0$, also der Punkt P von A aus gesehen links von der Strecke liegt: $\Rightarrow$ Testwert ist -1.



Wichtig: Zu beachten ist dabei noch, dass für abwärts laufende Segmente der Anfangspunkt der Strecke bei Überquerungen **ausgenommen** wird (Es muss also für eine Querung $y_A \neq y_P$ gelten.) und für aufwärts laufende Segmente der Endpunkt der Strecke (Es muss also für eine Querung $y_B \neq y_P$ gelten.). Verläuft die Überquerung in diesem oberen Endpunkt einer Strecke, so ist der Testwert Null.

Durch diese Regel wird vermieden, dass die Überquerungen in den Endpunkten der Strecken fälschlich doppelt gezählt werden.

Hinweis: Da unsere beiden Vektoren, von denen wir das Kreuzprodukt bilden, in der XY-Ebene liegen, zeigt das Ergebnis direkt in Z-Richtung und hat keine Komponenten in X- bzw Y-Richtung. Das Kreuzprodukt ist eine nützliche Sache: Der Betrag des Kreuzprodukts (in unserem einfach Fall der Betrag der Z-Komponente) liefert die von den beiden Operanden aufgespannte Parallelogrammfläche. Das Vorzeichen der Z-Komponente definiert dabei den für uns hier wichtigen Umlaufsinn.

Schließlich sind die Testwerte für alle Strecken des Polygonzugs für einen gegebenen Punkt P aufzuaddieren. Liefert das Resultat einen von Null verschiedenen Wert, liegt der Punkt innerhalb des Polygons, andernfalls außerhalb.

Hinweis: Falls Sie sich fragen, ob der Rand des Polygons mit zur Fläche zu zählen ist: Mathematisch gesehen ist die Frage irrelevant, weil der Rand die Fläche Null hat. Da wir hier in der realen digitalen Implementierung keinen Rand der Breite Null haben, ist die Frage aber dennoch berechtigt.

Die Unschärfe, die durch diese Fragestellung entsteht, ist allerdings (außer bei sehr kleinen Polygonen oder solchen mit sehr kleiner Innenfläche) sehr gering, da die Länge des Rands linear wächst und die davon umschlossene Fläche ca. quadratisch. Daher lassen wir die Frage offen.

2.3 Das Gesamtbild

Der Monte-Carlo-Algorithmus zur Integration der polygonbegrenzten Fläche A_P wird also folgendermaßen implementiert:

1. Umschließende Fläche definieren, wie in Abschnitt 2.2.1 beschrieben.
2. Festlegen einer Anzahl von Testpunkten. Starten Sie mit 10% der Punkte im gefundenen Rechteck K .
3. Die definierte Anzahl von Testpunkten wird getestet.
 - Hierbei wird jeweils ein zufälliger Punkt $P_i = (x, y)$ innerhalb des Rechtecks bestimmt, indem zwei zufällige Koordinaten x und y aus dem entsprechenden Bereich ermittelt werden.

Hinweis: Sie müssen *keine* Vorkehrungen dagegen treffen, den selben Punkt zufällig mehrfach zu wählen. Das darf ruhig passieren.
 - Sodann wird ermittelt, ob der Punkt P innerhalb oder außerhalb der durch den Polygonzug begrenzten Fläche liegt (siehe Abschnitt 2.2.2).

- Abhängig vom Ergebnis der Prüfung wird gezählt, wie viele Testpunkte innerhalb Z_i oder außerhalb Z_a des Polygonzugs zu liegen kommen.

4. Nach Abschluss aller Versuche, ist das Flächenverhältnis

$$\frac{A_P}{A_K} \quad (1)$$

gegeben durch

$$\frac{A_P}{A_K} = \frac{Z_i}{Z_i + Z_a} \quad (2)$$

da alle Versuche im Rechteck K liegen, aber nur Z_i Versuche in der Polygonfläche. Da die Fläche des Rechtecks leicht zu berechnen und daher bekannt ist, kann so die Polygonfläche berechnet werden.

Wichtig: Der Einfachheit halber sollen alle Berechnungen in der Längeneinheit Pixel ausgeführt werden. Damit erhalten Sie die Fläche am Ende in der Einheit Quadratpixel.

3 Arbeitsauftrag

3.1 Test der Grafikbibliothek

Zum Zeichnen des Polygons steht eine minimale BMP-Bibliothek zur Verfügung, welche Sie in Ihr Programm einbinden müssen.

Da „richtige“ Grafikbibliotheken zwar sehr mächtig sind, aber ihre Einbindung in eigene Projekte in der Regel auch einigen Konfigurationsaufwand in der Entwicklungsumgebung bedeutet, haben wir für Sie eine sehr rudimentäre Grafikbibliothek zum Erstellen von BMP-Dateien entwickelt. Diese unterstützt nur sehr wenige Zeichenoperationen und kann auch nur (unkomprimierte) BMP-Dateien erzeugen. Dafür müssen Sie zur Verwendung nur zwei Dateien in Ihr Projekt einbinden.

Die Bibliothek enthält die benötigten Funktionen sowie die Typdefinitionen `bmp` und `pixelcolor`. Die Grafik wird im Speicher durch eine Variable vom Typ `bmp` verwaltet und kann durch `save_BMP` in eine BMP-Datei geschrieben werden.

Zum Betrachten der Grafik können Sie dann ein geeignetes Grafikprogramm auf Ihrem Rechner benutzen.

Jede Variable vom Typ `bmp` stellt dabei eine eigene Zeichnung dar und muss über einen Aufruf der Funktion

```
bmp newBMP(long dx, long dy);
```

initialisiert werden, bevor mit ihr gearbeitet werden kann. Dadurch werden auch die Abmessungen der Grafik in Pixeln in x-Richtung und y-Richtung festgelegt.

Danach können Sie mit den weiteren Funktionen die Grafik bearbeiten bzw. in eine Datei schreiben.

```
void BMPclear(bmp image, pixelcolor color);
```

Hiermit löschen Sie die Grafik mit der beabsichtigten Hintergrundfarbe. Ihnen stehen folgende Konstanten für die Auswahl der Zeichenfarbe zur Verfügung:

```
extern const pixelcolor COL_black;  
extern const pixelcolor COL_white;  
extern const pixelcolor COL_green;  
extern const pixelcolor COL_red;  
extern const pixelcolor COL_blue;  
extern const pixelcolor COL_yellow;  
extern const pixelcolor COL_cyan;  
extern const pixelcolor COL_purple;  
extern const pixelcolor COL_magenta;
```

die folgenden Funktionen dienen zum zeichnen einzelner Punkte oder Linien.

```
void setPixel(bmp image, int x, int y,  
             pixelcolor color);  
void drawLine(bmp image, int x1, int y1,  
             int x2, int y2,  
             pixelcolor color);
```

Der Koordinatenursprung für diese Zeichenbefehle, also der Pixel mit den Koordinaten (0,0), liegt **links unten** in der Grafik. Das bedeutet also, das erzeugte Bild stellt den I. Quadranten eines kartesischen Koordinatensystems dar. Für die Angabe der Farbe können Sie wieder die oben aufgelisteten Farbkonstanten verwenden.

```
void save_BMP(char *name, bmp image);
```

Schließlich sichert das Bild in eine Datei, so dass Sie es mit einem Grafikprogramm betrachten können. Damit die Grafik richtig erkannt wird, ist es oft hilfreich, ihr die Dateiendung **.bmp** zu geben.

Bringen Sie zunächst das Beispielprogramm auf Ihrem System zum Laufen. Hierfür entpacken Sie das Archiv aus dem Onlinekurs, erstellen ein Projekt, welches die drei Dateien enthält und übersetzen es. Achten Sie darauf, den Dateinamen für den BMP-Export nach Ihren Vorstellungen abzuändern. Sie sollten eine Datei wie in Abbildung 1 erhalten.



Compilieren Sie das Beispielprogramm für die Grafikbibliothek.

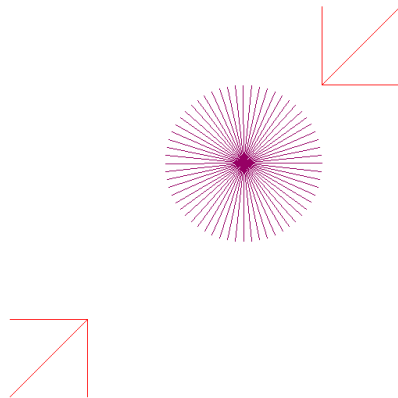


Abbildung 1: Die Grafik, wie sie das Beispielprogramm erzeugt.

3.1.1 Das Polygon zeichnen

Im ersten Schritt sollen Sie nun das Polygon zeichnen. Das Polygon ist durch ein Array gegeben, welches die Eckpunkte des Polygonzugs als Strukturdaten enthält, die ihrerseits aus x - und y -Koordinaten bestehen.

Definieren Sie zunächst einen Strukturdatentyp

```
struct Point
{
    int x;
    int y;
};
```

und verwenden Sie dann die Definition des Polygons aus der zur Verfügung gestellten Datei `polygon.snip`, indem Sie sie in die `main`-Funktion Ihres Programms kopieren.

Wichtig: Fügen Sie die Definition des Polygons nicht als globale Variable in Ihr Programm ein.

Zeichnen Sie nun das Polygon.

Ändern Sie hierfür das Beispielprogramm ab, indem Sie die Beispielgrafik entfernen und eine Funktion entwickeln, welche das Polygon zeichnet.

Dabei soll die Definition der Grafik, sowie die Ausgabe in die Datei in der `main`-Funktion verbleiben. Die Funktion zum Zeichnen des Polygons soll als Parameter einen Pointer auf die Grafik, die Punkteliste des Polygonzugs sowie die Anzahl der Punkte des Polygonzugs erhalten.

Hinweis: Es ist sinnvoll, die Grafik mit einer Größe von z.B. 1600x900 Punkten zu erzeugen – Sie können die Grafikgröße aber auch passender wählen.



Zeichnen Sie das vorgegebene Polygon in die BMP-Datei.



Testen Sie Ihre Implementierung!

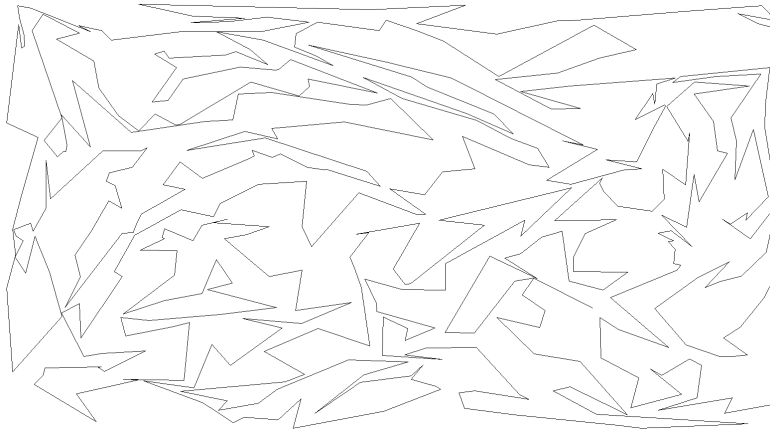


Abbildung 2: Das gezeichnete Polygon in einer Grafik mit 1600x900 Pixel.

Testen Sie nun Ihre Implementierung, indem Sie die Grafik in eine Datei ausgeben lassen und auf Korrektheit prüfen. In Abbildung 2 sehen Sie das erwünschte Ergebnis.

3.1.2 Das Rechteck

Implementieren Sie die Berechnung des Rechtecks, welches das Polygon einschließt als eigenständige Funktion.

Diese soll die Liste der Eckpunkte des Polygons, die Anzahl der Eckpunkte sowie zwei Pointer auf Variablen des Typs `struct Point` als Parameter erhalten. Die beiden letztgenannten Parameter dienen zur Rückgabe der linken unteren Ecke, sowie der rechten oberen Ecke des umschließenden Rechtecks.

Implementieren Sie nun eine weitere Funktion. Sie soll in eine per Pointer übergebene `bmp`-Variable (erster Parameter) ein Rechteck zeichnen, das durch zwei weitere Parameter vom Typ `struct Point` gegeben ist (linke untere Ecke sowie rechte obere Ecke). Sinnvollerweise zeichnen Sie das Rechteck in einer anderen Farbe als den Polygonzug.

Hinweis: Zum Zeichnen des Rechtecks können Sie natürlich die Polygon-Funktion verwenden, die Sie auch schon zum Zeichnen des Polygons verwendet haben, schließlich ist das Rechteck nur ein spezielles Polygon.

Verwenden Sie die beiden Funktionen, um das Rechteck zu berechnen und zu zeichnen und geben Sie die Grafik in eine Datei aus, um das Ergebnis zu prüfen (vgl. Abbildung 3).

Wichtig: Beheben Sie nun unbedingt alle eventuell vorhandenen Fehler bevor Sie weiter fortfahren.



Implementieren Sie eine Funktion zum Berechnen des umschließenden Rechtecks.



Implementieren Sie eine Funktion zum Zeichnen eines Rechtecks.



Testen Sie nun Ihre Implementierungen.

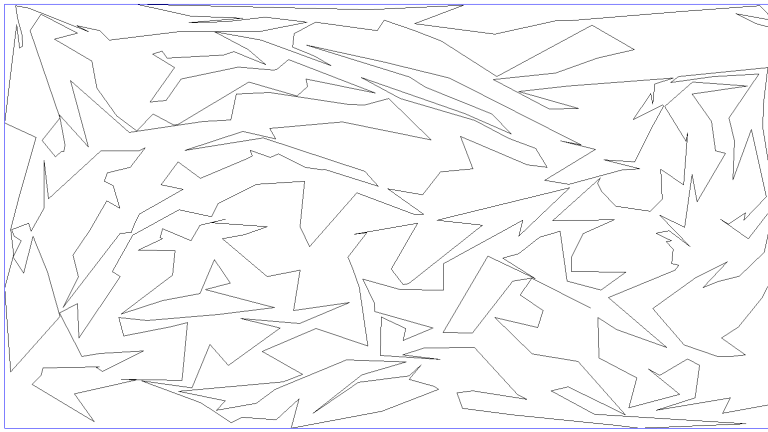


Abbildung 3: Gezeichnetes Rechteck, welches das Polygon einschließt.

3.1.3 Innenraumerkennung

Schreiben Sie eine Funktion, welche mit Hilfe der Methode des Jordan-Tests (siehe Abschnitt 2.2.2) berechnet, ob ein als Parameter übergebener Punkt im Inneren des Polygonzugs liegt.

Die Funktion soll als weitere Parameter die Liste der Polygon-Eckpunkte und deren Anzahl erhalten.

Als Rückgabewert soll die Funktion einen Wahrheitswert liefern, der angibt, ob der Punkt im Inneren liegt (wahr) oder nicht (unwahr).

Hinweis: Es empfiehlt sich, für den Test einer einzelnen Strecke eine Funktion zu entwickeln und diese dann in einer weiteren Funktion für alle Polygonsegmente aufzurufen und die Summation der Testergebnisse der Einzelsegmente vorzunehmen. Mit Hilfe dieser Summe kann dann die Bewertung des Testpunktes P vorgenommen und der geforderte Rückgabewert berechnet werden.



Schreiben Sie eine Funktion, die einen Punkt auf Lage im Inneren des Polygonzugs prüft.

3.1.4 Die Monte-Carlo-Flächenberechnung

Nun haben Sie alle nötigen Vorarbeiten ausgeführt, um den Algorithmus anzupacken.

Implementieren Sie den Monte-Carlo-Algorithmus wie eingangs beschrieben.

Hinweis: Falls Sie noch nicht bzw. nicht mehr wissen, wie man in C Zufallszahlen generieren kann, hier das Wesentliche in Kürze:

Die Standardbibliothek enthält (über den Header `stdlib.h`) einen Generator für Pseudozufallszahlen. Dies sind keine wirklich zufälligen Zahlen, sondern es werden Zahlen geliefert, die eine Verteilung aufweisen die der einer Folge zufälliger Zahlen gleicht. Dabei wird aus einem Startwert (engl. seed) bei jedem Abruf einer Zufallszahl



Implementieren Sie nun die Flächenberechnung nach der Monte-Carlo-Methode.

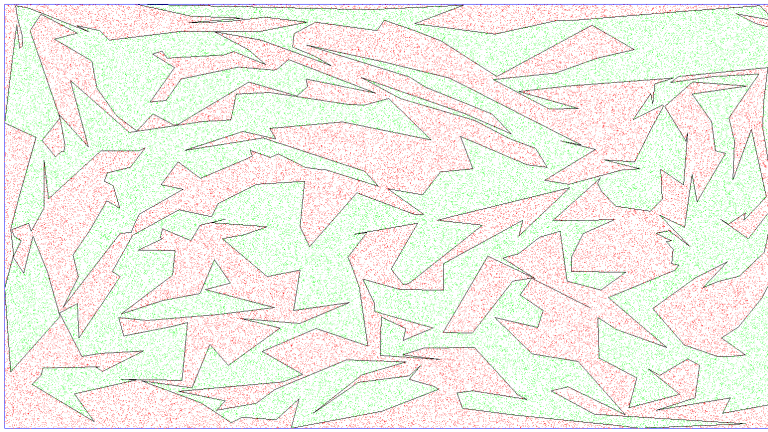


Abbildung 4: Ergebnis des Monte-Carlo-Experiments. Die unterschiedliche Farbgebung der Punkte zeigt die korrekte Erkennung von Innen- und Außenbereich des Polygonzugs.

jeweils ein neuer seed-Wert berechnet und daraus auch die „Zufallszahl“ abgeleitet.

Wählt man immer den gleichen Seed-Wert, den man mittels der Funktion `srand` setzt, der hierfür ein `unsigned`-Wert für den Startwert übergeben wird, dann erhält man auch immer die gleiche Zahlenfolge. Letztlich handelt es sich also hier gewissermaßen um „eingefrorenen“ Zufall. Lässt man den Aufruf von `srand` weg, wird der Generator mit dem Seed-Wert 1 initialisiert. Um zu vermeiden, dass das Programm nach dem Start immer mit der selben Zahlenfolge arbeitet, sollte man die über `time(NULL)` (in der Header-Datei `time.h`) abrufbare Systemzeit als Startwert des Generators verwenden.

Zufallszahlen erhält man nach der (einmaligen!) Initialisierung dann über wiederholte Aufrufe der Funktion `rand`, welche einen `unsigned`-Wert im Bereich $[0, \text{RANDMAX}[$ zurückliefert (und den internen Seed-Wert aktualisiert).

Eine von `rand` gelieferte Zufallszahl kann dann z.B. über Modulo-Operationen und Additionen auf den gewünschten Int-Bereich oder durch Division durch `RAND_MAX` auf den Fließkomma-Bereich $[0, 1[$ abgebildet werden.

Zeichnen Sie alle zufällig ermittelten Punkte mit `setPixel` in die Grafik ein. Verwenden Sie dabei unterschiedliche Farben für Punkte innerhalb bzw. außerhalb des Polygons.

Abbildung 4 zeigt ein Beispiel für eine korrekte Implementierung.

Berechnen Sie die Fläche des Polygonzugs und geben Sie alle relevanten Werte wie folgt auf dem Bildschirm aus:



Geben Sie die Ergebnisse der Flächenberechnung auf dem Bildschirm aus.

```
Zi:          XXXXX
Za:          XXXXX
Verhaeltnis: X.XXXXXX
-----
Rechteck: XXXXXX Quadratpixel
Polygon:  XXXXXX Quadratpixel
```

Frage: Führen Sie das Programm häufiger aus und vergleichen Sie die Ergebnisse.

Wie stabil sind die Ergebnisse?

Experimentieren Sie hierfür auch mit der Anzahl der zufälligen Punkte.

Wie viele Punkte benötigen Sie für ein akzeptables Ergebnis (Stabilität ca 10% oder 1%)?

Stellen Sie sicher, dass Sie diese Fragen bei der Abgabe diskutieren können!

Literatur

- [1] Juraj Hromkovič: Randomisierte Algorithmen. Methoden zum Entwurf von zufallsgesteuerten Systemen für Einsteiger. 1. Auflage. Teubner, 2004, ISBN 3-519-00470-4.
- [2] Wikipedia: Punkt-in-Polygon-Test nach Jordan,
https://de.wikipedia.org/wiki/Punkt-in-Polygon-Test_nach_Jordan.
- [3] Daniel Sunday: Practical Geometry Algorithms
https://geomalgorithms.com/a03-_inclusion.html und
https://web.archive.org/web/20130126163405/http://geomalgorithms.com/a03-_inclusion.html.
- [4] Wikipedia: Jordanscher Kurvensatz,
https://en.wikipedia.org/wiki/Jordan_curve_theorem